



COURSE SLIDES · WSQ

CompTIA Certified Linux+ Training (XK0-006)

WSQ Course Code: TGS-2024048316 · CompTIA Linux+ XK0-006 V8

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Trainer: Dr. Alfred Ang · Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE

Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

CERTIFICATION

Linux / cloud / DevOps & cybersecurity — hands-on Linux systems administration.

DELIVERS

WSQ courses on Linux, cloud administration, DevOps and software engineering.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

- Your name and organisation / role.
- Your current experience with Linux (beginner → advanced).
- Which of the five exam domains matters most to your day job, and your exam timeline.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

Do every lab yourself on Killercode.

5

Be punctual; return from breaks on time.

6

75% attendance is required.

LMS / TMS

- Access your course materials, attendance and assessment on the LMS/TMS portal.
- Portal: <https://lms-tms.tertiaryinfotech.com/>
- Download the slides and Learner Guide for reference during the open-book assessment.

SCHEDULE

Lesson Plan — 2 Days, 8 hours/day

Day 1

- Day 1 — System & User Management
 - Domain 1: System Management, 23% (Labs 1–7)
 - Domain 2: Services & User Management, 20% (Labs 8–13)
 - 9:00am–6:00pm · 1-hour lunch · tea within

Day 2

- Day 2 — Security, Automation & Troubleshooting
 - Domain 3: Security, 18% (Labs 14–19)
 - Domain 4: Automation & Scripting, 17% (Labs 20–24)
 - Domain 5: Troubleshooting, 22% (Labs 25–30)
 - Final Assessment (WA + PP) from 4:00pm

Learning Outcomes

1

System Management

Boot & FHS, kernel modules, LVM storage, networking, shell, backup, virtualization.

2

Services & User Management

Files, accounts, processes, packages, systemd services, containers.

3

Security

AAA/sudo/PAM, firewalls, OS & account hardening, cryptography, compliance & audit.

4

Automation & Scripting

Ansible IaC, Bash, Python, Git version control, responsible AI use.

5

Troubleshooting

Monitoring, and storage / network / security / performance triage.

BEFORE THE ASSESSMENT

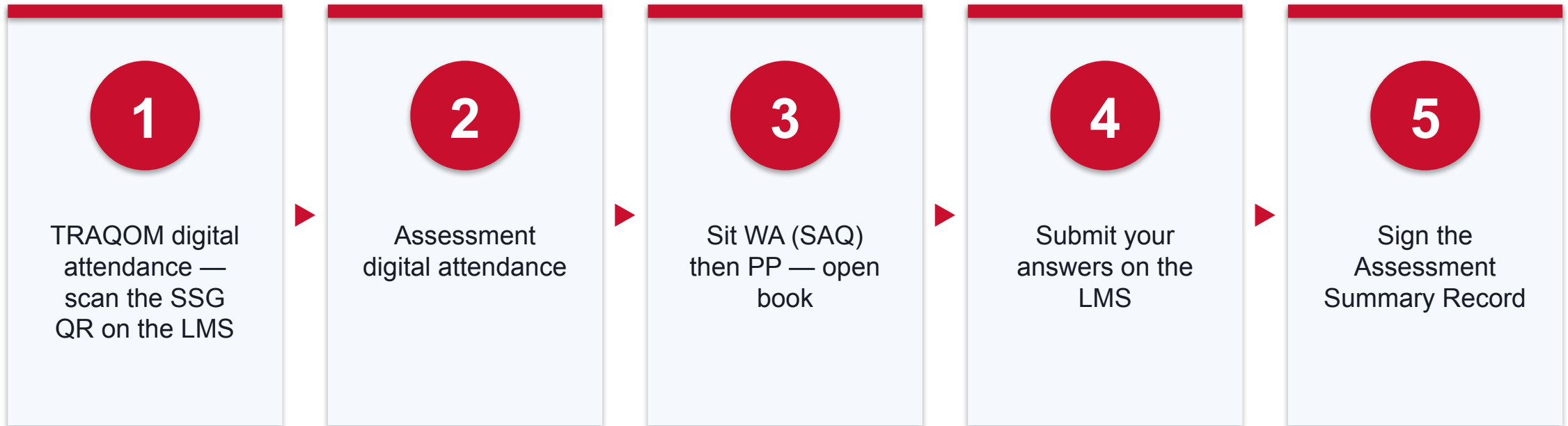
Briefing for Assessment

- Place phones and other materials under the table or on the floor.
- No photos or recording of assessment scripts.
- No discussion during the assessment.
- Use a black/blue pen for hard-copy assessments.
- No liquid paper / correction tape.
- Scripts are collected when time is up.

Assessment

- Written Assessment (WA) — Short-Answer Questions (SAQ), 1 hour, open book — aligned to the slides.
- Practical Performance (PP) — hands-on Linux tasks, 1 hour, open book — aligned to the labs.
- Format: Open Book — slides, Learner Guide and approved materials only.
- A minimum of 75% attendance is required to be eligible for assessment and funding.
- An appeal process is available if required.

Assessment Flow





CORE CONCEPTS

Linux Fundamentals for Administrators

What is Linux?

1

Open-source OS

A free, open-source operating system kernel plus GNU userland, powering most servers and clouds.

2

Distributions

Debian/Ubuntu (apt/dpkg) and RHEL/Rocky/Fedora (dnf/rpm) package the kernel differently.

3

Many ways to manage

The shell (bash), config files under /etc, systemd, and automation with Ansible/Bash/Python.

4

The Linux+ role

Configure, manage, secure, automate and troubleshoot Linux servers.

The Linux Landscape

Kernel & FHS

- Kernel + initramfs boot userspace
- FHS: /etc /var /usr /home ...
- Everything is a file

Distributions

- Debian/Ubuntu — apt/dpkg
- RHEL/Rocky/Fedora — dnf/rpm
- Same kernel, different tooling

Architectures & licensing

- x86_64, ARM64 (AArch64), RISC-V
- Free / open-source vs proprietary
- Copyleft (GPL) licensing

WHY IT MATTERS

On Linux, everything is a file.

Files, directories, devices, sockets and processes are all reachable through the filesystem — master the shell and you master the system.

Ways to Work on Linux

Interactive

- The shell (bash)
 - Interactive commands + pipes
 - Fast and scriptable
- Text editors
 - vi/vim and nano
 - Edit config files in place

Automation

- Scripting & automation
 - Bash and Python scripts
 - Ansible for Infrastructure as Code
- Containers & services
 - systemd manages services
 - Docker/Podman run apps

Your Workbench — Killercoda

- Every lab runs free in the browser on the Killercoda Ubuntu Playground:
<https://killercoda.com/playgrounds/scenario/ubuntu>
- A disposable Ubuntu VM — install packages, break things and reset with no risk.
- Type the commands yourself; muscle memory is what the exam tests.
- Reset the playground between labs that change kernel, firewall or systemd state.

 DOMAIN 01

System Management

Objectives 1.1–1.7 · 23% of the exam · Labs 1–7

Key Concepts — System Management

1

The boot chain: firmware/UEFI → GRUB2 → kernel + initramfs → systemd; the FHS defines where files live.

2

Storage stacks up: partitions → LVM (PV → VG → LV) → filesystem (ext4/xfs/btrfs), mounted via /etc/fstab.

3

The network stack is read top-down: NetworkManager/Netplan config → ip/ss → DNS resolution → diagnostics.

4

Shell operations (redirection, pipes, text tools), backup (tar/rsync/dd) and virtualization (QEMU/KVM) round out the domain.

Hands-On Labs — System Management

Labs 1–3

- Boot Process & Filesystem Hierarchy
- Kernel Modules & Device Management
- Storage with LVM, Partitions & Filesystems

Labs 4–6

- Network Configuration
- Shell Operations & Text Processing
- Backup & Restore

Lab 7

- Virtualization with QEMU/KVM & libvirt

Boot Process & Filesystem Hierarchy

LAB 1

Inspect a running Linux system's boot configuration, kernel command line, initramfs, and the standard Filesystem Hierarchy to understand how GRUB hands off to the kernel and where FHS directories live.

You'll build

Identify the distro and kernel, inspect GRUB, the kernel cmdline, and initramfs, then tour the FHS and boot-related directories.

Key commands: `/etc/os-release`, `GRUB2`, `/proc/cmdline`, `initramfs`, `lsinitramfs`, `FHS`

Boot Process & Filesystem Hierarchy

STEP 1 OF 6

1

Identify distribution and architecture

```
$ cat /etc/os-release
```


Boot Process & Filesystem Hierarchy

STEP 2 OF 6

2

Inspect GRUB configuration

```
$ cat /etc/default/grub 2>/dev/null || true
```

Boot Process & Filesystem Hierarchy

STEP 3 OF 6

3

Inspect the kernel command line

```
$ cat /proc/cmdline
```

Boot Process & Filesystem Hierarchy

STEP 4 OF 6

4

Explore the initramfs

```
$ ls /boot/initrd* 2>/dev/null || echo "no initrd image present"
```

Boot Process & Filesystem Hierarchy

STEP 5 OF 6

5

Tour the Filesystem Hierarchy (FHS)

```
$ for d in /bin /boot /dev /etc /home /lib /proc /sbin /sys /tmp /usr /var /run /opt /srv;
```

Boot Process & Filesystem Hierarchy

STEP 6 OF 6

6

Walk through key boot-related dirs

```
$ ls /etc/systemd/system/ | head
```

Boot Process & Filesystem Hierarchy

Test it

You can identify the running distro/kernel/arch and explain how GRUB, the kernel cmdline, and initramfs bring up userspace.

Kernel Modules & Device Management

LAB 2

List, inspect, load, and unload kernel modules and use the standard hardware-enumeration tools to understand modprobe vs insmod, module dependencies, and reading dmesg.

You'll build

Install discovery tools, inspect and load/unload a module, examine module dependencies, read the kernel ring buffer, and enumerate hardware.

Key commands: lsmod, modinfo, modprobe, insmod/rmmod, depmod, dmesg

Kernel Modules & Device Management

STEP 1 OF 7

1

Install hardware inspection tools

```
$ apt-get update -qq
```


Kernel Modules & Device Management

STEP 2 OF 7

2

List loaded modules and inspect one

```
$ lsmod | head -20
```

Kernel Modules & Device Management

STEP 3 OF 7

3

Load and unload a module

```
$ modprobe dummy 2>&1 || modprobe loop 2>&1 || echo "module load blocked"
```

Kernel Modules & Device Management

STEP 4 OF 7

4

Module dependencies and depmod

```
$ cat /lib/modules/$(uname -r)/modules.dep 2>/dev/null | head -5
```

Kernel Modules & Device Management

STEP 5 OF 7

5

Read the kernel ring buffer

```
$ dmesg | tail -20 2>/dev/null || journalctl -k --no-pager | tail -20
```

Kernel Modules & Device Management

STEP 6 OF 7

6

Enumerate hardware

```
$ lscpu
```

Kernel Modules & Device Management

STEP 7 OF 7

7

initramfs builders overview

```
$ which dracut 2>/dev/null || echo "dracut on RHEL/Fedora/SUSE"
```

Kernel Modules & Device Management

Test it

You can pick the right ls* tool for a hardware question and explain insmod vs modprobe vs depmod.

Storage with LVM, Partitions & Filesystems

LAB 3

Build a complete LVM stack on loopback-backed disks, format with ext4/xfs/btrfs, mount with hardening options, persist in fstab, and grow a filesystem online.

You'll build

Create loopback disks, partition, build PVs/VG/LVs, make filesystems, mount with hardening options into fstab, then extend an LV and its filesystem live.

Key commands: `losetup`, `parted/GPT`, `pvcreate/vgcreate/lvcreate`, `mkfs.ext4/xfs/btrfs`, `blkid/UUID`, `/etc/fstab`

Storage with LVM, Partitions & Filesystems

STEP 1 OF 6

1

Install tooling and create loopback disks

```
$ apt-get install -y lvm2 xfsprogs btrfs-progs parted util-linux >/dev/null
```

Storage with LVM, Partitions & Filesystems

STEP 2 OF 6

2

Partition the first loopback disk

```
$ LOOP1=$(losetup -j /lab/disks/disk1.img | cut -d: -f1)
```

Storage with LVM, Partitions & Filesystems

STEP 3 OF 6

3

Create PVs, a VG, and LVs

```
$ LOOP2=$(losetup -j /lab/disks/disk2.img | cut -d: -f1)
```

Storage with LVM, Partitions & Filesystems

STEP 4 OF 6

4

Make filesystems

```
$ mkfs.ext4 -F /dev/labvg/lv_ext4
```

Storage with LVM, Partitions & Filesystems

STEP 5 OF 6

5

Mount with hardening options and persist in fstab

```
$ mkdir -p /mnt/ext4 /mnt/xfs /mnt/btrfs
```

Storage with LVM, Partitions & Filesystems

STEP 6 OF 6

6

Grow a logical volume and its filesystem online

```
$ lvextend -L +100M /dev/labvg/lv_ext4
```

Storage with LVM, Partitions & Filesystems

Test it

You can go from raw block device to a mounted, UUID-referenced, online-expandable filesystem; note XFS grows but cannot shrink.

Network Configuration

LAB 4

Inspect interfaces, addresses, and routes with `ip`; walk the name-resolution stack; validate a netplan YAML; and use the standard diagnostic toolbox to read a Linux network stack top-to-bottom.

You'll build

Install network tools, inspect interfaces/routes, walk the resolution stack, run DNS diagnostics, validate netplan, check sockets/connectivity, and capture packets/scan ports.

Key commands: `ip -br link/address/route`, `/etc/nsswitch.conf`, `/etc/resolv.conf`, `getent`, `dig/nslookup`, `netplan generate`

Network Configuration

STEP 1 OF 7

1

Install tools

```
$ apt-get update -qq
```

Network Configuration

STEP 2 OF 7

2

Interfaces, addresses, routes

```
$ ip -br link
```

Network Configuration

STEP 3 OF 7

3

Name resolution stack

```
$ cat /etc/hosts
```

Network Configuration

STEP 4 OF 7

4

DNS diagnostics

```
$ dig +short killercoda.com
```

Network Configuration

STEP 5 OF 7

5

NetworkManager and netplan syntax

```
$ nmcli device status 2>&1 | head
```

Network Configuration

STEP 6 OF 7

6

Sockets and connectivity tests

```
$ ss -tulnp | head
```

Network Configuration

STEP 7 OF 7



Packet capture and port scan

```
$ timeout 3 tcpdump -i any -nn -c 5 'icmp or port 53' 2>&1 &
```

Network Configuration

Test it

You can read the resolution path (nsswitch -> hosts/resolv.conf -> DNS) and pick the right diagnostic: ss, dig/getent, tcpdump, nmap.

Shell Operations & Text Processing

LAB 5

Work with shell environment variables, login vs interactive startup files, every common redirection form, and the classic text-processing pipeline to assemble non-trivial one-liners.

You'll build

Explore env vars and PATH, paths and startup files, all redirection operators, then grep/cut/sort/uniq, sed/awk, tr/xargs, and scripted editors.

Key commands: export/env/unset, login vs interactive shells, redirection > >> < << <<< | tee, grep/cut/sort/uniq/wc, sed/awk, tr/xargs

Shell Operations & Text Processing

STEP 1 OF 7

1

Environment variables and PATH

```
$ echo "USER=$USER HOME=$HOME SHELL=$SHELL"
```

Shell Operations & Text Processing

STEP 2 OF 7

2

Absolute vs relative paths and startup files

```
$ mkdir -p /tmp/lab5 && cd /tmp/lab5
```

Shell Operations & Text Processing

STEP 3 OF 7

3

Redirection operators

```
$ echo "first line" > out.txt
```

Shell Operations & Text Processing

STEP 4 OF 7

4

grep, cut, sort, uniq, wc

```
$ cp /etc/passwd .
```

Shell Operations & Text Processing

STEP 5 OF 7

5

sed and awk

```
$ sed -n '1,5p' passwd
```

Shell Operations & Text Processing

STEP 6 OF 7

6

head, tail, tr, xargs

```
$ head -3 passwd
```

Shell Operations & Text Processing

STEP 7 OF 7

7

Editors: nano and vim, scripted

```
$ apt-get install -y vim nano >/dev/null
```


Shell Operations & Text Processing

Test it

You can compose grep/awk/sed/cut/sort/uniq/tr/xargs into ad-hoc pipelines and know which startup file each shell type sources.

Backup & Restore

LAB 6

Create tar/cpio/dd archives with gzip/bzip2/xz compression, image and recover disks with dd and ddrescue, mirror trees with rsync, and verify integrity with sha256sum.

You'll build

Make sample data, archive with tar and cpio, dd a disk image, recover a damaged one with ddrescue, mirror with rsync --delete, then verify checksums and read compressed files in place.

Key commands: tar (czf/xzf/tzf), cpio, dd, ddrescue, rsync --delete, sha256sum -c

Backup & Restore

STEP 1 OF 7

1

Install tooling and create sample data

```
$ apt-get install -y tar cpio gzip bzip2 xz-utils rsync gddrescue coreutils >/dev/null
```

Backup & Restore

STEP 2 OF 7

2

tar with gzip, bzip2, and xz

```
$ cd /lab6
```

Backup & Restore

STEP 3 OF 7

3

cpio (the older archiver)

```
$ cd /lab6
```

Backup & Restore

STEP 4 OF 7

4

dd a disk image to a loopback file

```
$ cd /lab6
```

Backup & Restore

STEP 5 OF 7

5

ddrescue from a damaged image

```
$ cd /lab6
```

Backup & Restore

STEP 6 OF 7

6

rsync local-to-local with delete

```
$ mkdir -p /lab6/mirror
```


Backup & Restore

STEP 7 OF 7

7

Integrity and reading compressed files

```
$ cd /lab6
```

Backup & Restore

Test it

You can choose tar/cpio/dd/ddrescue/rsync appropriately and know sha256sum -c and rsync trailing-slash/--delete semantics.

Virtualization with QEMU/KVM & libvirt

LAB 7

Install QEMU and libvirt, manipulate virtual disk images with `qemu-img`, list libvirt domains/networks with `virsh`, and boot a tiny guest under QEMU to discuss hypervisors, image formats, snapshots, and network modes.

You'll build

Install QEMU/libvirt, create and inspect disk images, convert/resize them, take qcow2 snapshots, list libvirt domains/networks, boot a guest under TCG, and review the three network modes.

Key commands: QEMU vs KVM, `/dev/kvm` vs TCG, `qemu-img` (qcow2/raw), `qemu-img convert/resize`, qcow2 snapshots, `virsh list/net-list`

Virtualization with QEMU/KVM & libvirt

STEP 1 OF 7

1

Install QEMU and libvirt

```
$ apt-get install -y qemu-system-x86 qemu-utils libvirt-clients libvirt-daemon-system brid
```

Virtualization with QEMU/KVM & libvirt

STEP 2 OF 7

2

Create and inspect disk images

```
$ mkdir -p /lab7 && cd /lab7
```

Virtualization with QEMU/KVM & libvirt

STEP 3 OF 7

3

Convert and resize images

```
$ cd /lab7
```

Virtualization with QEMU/KVM & libvirt

STEP 4 OF 7

4

Snapshots inside a qcow2

```
$ cd /lab7
```

Virtualization with QEMU/KVM & libvirt

STEP 5 OF 7

5

libvirt: domains and networks

```
$ systemctl start libvirtd 2>/dev/null || service libvirtd start 2>/dev/null || true
```


Virtualization with QEMU/KVM & libvirt

STEP 6 OF 7

6

Boot a minimal guest under QEMU (TCG fallback)

```
$ cd /lab7
```

Virtualization with QEMU/KVM & libvirt

STEP 7 OF 7

7

Discuss network modes (concept)

\$ Bridged : guest is a peer on the LAN with its own DHCP IP.

Virtualization with QEMU/KVM & libvirt

Test it

You can distinguish QEMU (emulator) from KVM (accelerator), qcow2 from raw, internal vs libvirt snapshots, and the three guest network modes.

Recap — System Management

- Lab 1: Boot Process & Filesystem Hierarchy
- Lab 2: Kernel Modules & Device Management
- Lab 3: Storage with LVM, Partitions & Filesystems
- Lab 4: Network Configuration
- Lab 5: Shell Operations & Text Processing
- Lab 6: Backup & Restore
- Lab 7: Virtualization with QEMU/KVM & libvirt

Tea Break

15 minutes

 DOMAIN 02

Services and User Management

Objectives 2.1–2.6 · 20% of the exam · Labs 8–13

Key Concepts — Services and User Management

1

Everything is a file: manage files/links/devices, and local accounts via `/etc/passwd`, `/etc/shadow` and `/etc/group`.

2

Processes have states, priorities and signals; jobs are scheduled with `at`, `cron` and `anacron`.

3

Software is installed with `apt/dpkg` and `dnf/rpm`, plus language (pip/npm/cargo) and sandboxed (snap/flatpak) managers.

4

`systemd` manages services, timers, mounts and targets; containers (Docker/Podman) package and isolate applications.

Hands-On Labs — Services and User Management

Labs 8–9

- File & Directory Management
- Local Account & Group Management

Labs 10–11

- Processes, Jobs & Scheduling
- Software & Package Management

Labs 12–13

- Service Management with systemd
- Containers with Docker & Podman

File & Directory Management

LAB 8

Practice core file and directory operations: navigating, creating, copying, searching, linking, and inspecting special device files with standard Linux utilities.

You'll build

Create a lab tree, then copy, find, link, diff, and inspect files and /dev device nodes.

Key commands: ls -la / stat / file, find predicates (-name, -mtime, -size), locate & updatedb, lsof, diff & sdiff, hard vs symbolic links

File & Directory Management

STEP 1 OF 7

1

Navigate and list

```
$ pwd
```

File & Directory Management

STEP 2 OF 7

2

Create, copy, move, touch

```
$ touch src/file1.txt src/file2.log
```

File & Directory Management

STEP 3 OF 7

3

Find files by name, time, and size

```
$ dd if=/dev/zero of=src/big.bin bs=1M count=5
```

File & Directory Management

STEP 4 OF 7

4

locate and lsof

```
$ apt update -y && apt install -y mlocate lsof
```

File & Directory Management

STEP 5 OF 7

5

diff and sdiff

```
$ echo -e "alpha\nbravo\ncharlie" > a.txt
```

File & Directory Management

STEP 6 OF 7

6

Hard links vs symbolic links

```
$ echo "original" > target.txt
```

File & Directory Management

STEP 7 OF 7

7

Device files in /dev

```
$ ls -l /dev/sda /dev/null /dev/tty1 2>/dev/null
```


File & Directory Management

Test it

ls -li confirms hardlink shares the target's inode while the symlink breaks after the target is removed.

Local Account & Group Management

LAB 9

Create and manage local users and groups, set passwords and aging policies, change shells, and distinguish user, system, and service accounts.

You'll build

Create users/groups, set passwords and aging, inspect identity databases, then remove the accounts.

Key commands: `/etc/passwd, /etc/shadow, /etc/group, /etc/skel, useradd / adduser / usermod -aG, chpasswd, passwd -S, chage, chsh, getent, id, UID>=1000 user vs UID<1000 system vs service (-r), userdel -r / groupdel`

Local Account & Group Management

STEP 1 OF 7

1

Inspect account databases

```
$ head -5 /etc/passwd
```

Local Account & Group Management

STEP 2 OF 7

2

Create users and groups

```
$ groupadd developers
```

Local Account & Group Management

STEP 3 OF 7

3

Passwords and aging

```
$ echo "alice:Passw0rd!" | chpasswd
```

Local Account & Group Management

STEP 4 OF 7

4

Change shell and identify accounts

```
$ chsh -s /bin/sh bob
```

Local Account & Group Management

STEP 5 OF 7

5

User vs system vs service accounts

```
$ awk -F: '$3 >= 1000 && $3 < 65534 {print "USER ", $1, $3}' /etc/passwd
```

Local Account & Group Management

STEP 6 OF 7

6

Login history and active sessions

```
$ last | head
```


Local Account & Group Management

STEP 7 OF 7

7

Remove users and groups

```
$ userdel -r bob
```

Local Account & Group Management

Test it

`getent passwd bob` returns nothing after `userdel -r`, confirming the account and home directory are gone.

Processes, Jobs & Scheduling

LAB 10

Inspect processes, manipulate shell jobs, adjust priorities, send signals, and schedule tasks with `at`, `cron`, and `anacron`.

You'll build

Examine processes via `ps`/`proc`/`strace`, control background jobs, `renice`, `signal`, and schedule jobs.

Key commands: `ps auxf` / `pstree` / `pidstat` / `mpstat`, `/proc/<PID>` and `ls -l /proc/<PID>/fd`, process states (R,S,D,Z,T) & `strace`, jobs: `&`, `Ctrl+Z`, `bg`, `fg`, `nohup`, `disown`, `nice` / `renice` priority, `kill` / `pkill` / `killall` signals

Processes, Jobs & Scheduling

STEP 1 OF 7

1

Inspect running processes

```
$ ps auxf | head -20
```

Processes, Jobs & Scheduling

STEP 2 OF 7

2

Explore /proc and lsof

```
$ sleep 600 &
```

Processes, Jobs & Scheduling

STEP 3 OF 7

3

Process states and strace

```
$ ps -o pid,stat,cmd -p $SPID
```

Processes, Jobs & Scheduling

STEP 4 OF 7

4

Background, foreground, jobs

```
$ sleep 300 &
```

Processes, Jobs & Scheduling

STEP 5 OF 7

5

Priority with nice and renice

```
$ nice -n 10 sleep 200 &
```


Processes, Jobs & Scheduling

STEP 6 OF 7

6

Signals: kill, pkill, killall

```
$ sleep 1000 &
```

Processes, Jobs & Scheduling

STEP 7 OF 7

7

Scheduling with at and cron

```
$ service atd start 2>/dev/null || systemctl start atd
```

Processes, Jobs & Scheduling

Test it

atq lists the queued at job and crontab -l shows the recurring entry, confirming both schedulers accepted the tasks.

Software & Package Management

LAB 11

Manage Debian packages with apt/dpkg, add a signed third-party repo, run dnf/rpm in a container, and use language and sandboxed package managers.

You'll build

Install/remove apt packages, add a GPG-signed repo, demo dnf/rpm, then pip/npm/cargo and snap/flatpak.

Key commands: apt update/install/remove/purge/autoremove, apt-cache policy, dpkg -l / -L, signed-by GPG keyrings, dnf / rpm -q / rpm -V, pip, npm -g, cargo, update-alternatives

Software & Package Management

STEP 1 OF 7

1

apt basics

```
$ apt update
```

Software & Package Management

STEP 2 OF 7

2

Add a third-party repo with GPG key

```
$ apt install -y curl gnupg lsb-release
```

Software & Package Management

STEP 3 OF 7

3

Remove packages

```
$ apt remove -y tree
```

Software & Package Management

STEP 4 OF 7

4

RPM world inside a container

```
$ apt install -y docker.io
```


Software & Package Management

STEP 5 OF 7

5

Language-specific package managers

```
$ apt install -y python3-pip nodejs npm cargo
```

Software & Package Management

STEP 6 OF 7

6

update-alternatives

```
$ update-alternatives --display editor
```

Software & Package Management

STEP 7 OF 7



Sandboxed package formats

```
$ which snap && snap list 2>/dev/null || echo "snap not preinstalled on this image"
```

Software & Package Management

Test it

apt-cache policy terraform shows a candidate from the signed HashiCorp repo, confirming the GPG-verified source is trusted.

Service Management with systemd

LAB 12

Inspect and control systemd units, author custom service/timer/mount units, analyze boot performance, and configure host/time/DNS via systemd helpers.

You'll build

Manage nginx, write a service + timer + mount unit, analyze boot time, and set hostname/timezone/DNS.

Key commands: `systemctl list-units / list-unit-files / --failed`, `start/stop/restart/reload`, `enable/disable`, `mask/unmask`, custom `.service` (Type=oneshot) + `daemon-reload`, `.timer` units (OnUnitActiveSec), `.mount` units, `systemd-analyze blame / critical-chain`

Service Management with systemd

STEP 1 OF 7

1

List and inspect units

```
$ systemctl list-units --type=service | head -20
```

Service Management with systemd

STEP 2 OF 7

2

Control nginx

```
$ apt update -y && apt install -y nginx
```

Service Management with systemd

STEP 3 OF 7

3

Author a custom service

```
$ echo "lab-hello fired at $(date)" >> /var/log/lab-hello.log
```


Service Management with systemd

STEP 4 OF 7

4

Add a timer

\$ [Unit]

Service Management with systemd

STEP 5 OF 7

5

Mount unit example

```
$ mkdir -p /mnt/labdata
```

Service Management with systemd

STEP 6 OF 7

6

Boot performance

```
$ systemd-analyze
```

Service Management with systemd

STEP 7 OF 7

7

Host, time, and DNS

```
$ hostnamectl
```

Service Management with systemd

Test it

`systemctl list-timers` shows `lab-hello.timer` scheduled and `journalctl -u lab-hello` records each firing, confirming the custom units work.

Containers with Docker & Podman

LAB 13

Install Docker and Podman, pull/run images, manage volumes and networks, build an image from a Dockerfile, and contrast rootful vs rootless engines.

You'll build

Run and inspect nginx containers, bind-mount volumes, build an image, wire a user-defined network, and try rootless Podman.

Key commands: docker.io vs podman (daemonless/rootless), docker pull/run -d -p --name, docker ps, logs / exec -it / inspect / stats, bind-mount volumes (-v host:container:ro), Dockerfile: FROM/RUN/COPY/USER/ENTRYPOINT/CMD, docker network create (built-in DNS)

Containers with Docker & Podman

STEP 1 OF 7

1

Install Docker and Podman

```
$ apt update -y
```

Containers with Docker & Podman

STEP 2 OF 7

2

Pull and run a container

```
$ docker pull nginx
```


Containers with Docker & Podman

STEP 3 OF 7

3

Inspect, exec, logs

```
$ docker logs web | head
```

Containers with Docker & Podman

STEP 4 OF 7

4

Volumes for persistent data

```
$ mkdir -p /data
```

Containers with Docker & Podman

STEP 5 OF 7

5

Build a tiny image

```
$ mkdir -p /tmp/hello && cd /tmp/hello
```

Containers with Docker & Podman

STEP 6 OF 7

6

Networks and port mapping

```
$ docker network create labnet
```

Containers with Docker & Podman

STEP 7 OF 7

7

Rootless Podman and privileged note

```
$ podman run --rm alpine echo "hello from podman"
```

Containers with Docker & Podman

Test it

curl against the published ports returns 200 and the alpine container reaches `http://api` by name, confirming containers, volumes, and the user-defined network work.

Recap — Services and User Management

- Lab 8: File & Directory Management
- Lab 9: Local Account & Group Management
- Lab 10: Processes, Jobs & Scheduling
- Lab 11: Software & Package Management
- Lab 12: Service Management with systemd
- Lab 13: Containers with Docker & Podman

End of Day 1

See you tomorrow — 9:00 AM

DOMAIN 03

Security

Objectives 3.1–3.6 · 18% of the exam · Labs 14–19

Key Concepts — Security

1

AAA: authenticate (PAM/SSSD), authorize (sudo, Polkit) and account (journald, auditd) for every action.

2

Firewalls filter at every layer — ufw → firewalld → nftables/iptables — with stateful inspection and NAT.

3

OS & account hardening: permissions/ACLs, SELinux, SSH hardening, fail2ban, password policy and MFA.

4

Cryptography (GPG, LUKS2, OpenSSL, WireGuard) and compliance/audit (AIDE, rkhunter, OpenSCAP, CIS) protect data.

Hands-On Labs — Security

Labs 14–15

- AAA: sudo, PAM, and Polkit
- Firewalls: ufw, firewalld, iptables, nftables, ipset

Labs 16–17

- OS Hardening: Permissions, SELinux, SSH, Fail2ban
- Account Hardening

Labs 18–19

- Cryptography: GPG, LUKS, OpenSSL, WireGuard
- Compliance, Auditing, and File Integrity

AAA: sudo, PAM, and Polkit

LAB 14

Configure sudo authorization, inspect PAM authentication stacks, and enable auditd accounting to record security events. Finish with a conceptual look at centralized identity via LDAP, Kerberos, and SSSD.

You'll build

Install sudo, grant a user privileges, inspect PAM, read auth logs, and enable auditd rules.

Key commands: sudo / visudo / sudoers.d, PAM stacks (auth, account, session), auditctl / ausearch, journalctl / rsyslog / logrotate, pkexec (Polkit), SSSD / LDAP / Kerberos

AAA: sudo, PAM, and Polkit

STEP 1 OF 7

1

Install sudo and create a user

```
$ apt update && apt install -y sudo
```

AAA: sudo, PAM, and Polkit

STEP 2 OF 7

2

Grant sudo via group and drop-in file

```
$ usermod -aG sudo alice
```

AAA: sudo, PAM, and Polkit

STEP 3 OF 7

3

Switch identity with sudo and su

```
$ su - alice -c 'sudo -n systemctl status ssh || true'
```

AAA: sudo, PAM, and Polkit

STEP 4 OF 7

4

Inspect PAM stacks

```
$ ls /etc/pam.d/
```


AAA: sudo, PAM, and Polkit

STEP 5 OF 7

5

Read auth logs via journald and rsyslog

```
$ apt install -y rsyslog
```

AAA: sudo, PAM, and Polkit

STEP 6 OF 7

6

Enable auditd and watch /etc/passwd

```
$ apt install -y auditd audispd-plugins
```

AAA: sudo, PAM, and Polkit

STEP 7 OF 7

7

Persistent audit rules and Polkit note

```
$ -w /etc/passwd -p wa -k passwd_changes
```

AAA: sudo, PAM, and Polkit

Test it

`ausearch -k passwd_changes` returns the recorded write event, confirming accounting is active.

Firewalls: ufw, firewalld, iptables, nftables, ipset

LAB 15

Configure firewalls at every layer of the Linux netfilter stack, from the ufw wrapper down to iptables and nftables, and understand stateful inspection. Explore zones, rich rules, ipset, and SNAT/DNAT.

You'll build

Configure ufw, firewalld zones, nftables, ipset, NAT rules, and a stateful conntrack pattern.

Key commands: ufw allow/deny/enable, firewalld zones and rich rules, iptables / nftables chains, ipset hash:ip, SNAT / DNAT / MASQUERADE, conntrack states (NEW/ESTABLISHED/RELATED)

Firewalls: ufw, firewallld, iptables, nftables, ipset

STEP 1 OF 7

1

Install and configure ufw

```
$ apt update && apt install -y ufw
```

Firewalls: ufw, firewallld, iptables, nftables, ipset

STEP 2 OF 7

2

Explore iptables backing ufw

```
$ iptables -L -n -v
```

Firewalls: ufw, firewalld, iptables, nftables, ipset

STEP 3 OF 7

3

firewalld zones and rich rules in Rocky

```
$ apt install -y docker.io
```


Firewalls: ufw, firewalld, iptables, nftables, ipset

STEP 4 OF 7

4

nftables: table, chain, and a rule

```
$ apt install -y nftables
```

Firewalls: ufw, firewalld, iptables, nftables, ipset

STEP 5 OF 7

5

ipset for high-performance IP lists

```
$ apt install -y ipset
```

Firewalls: ufw, firewalld, iptables, nftables, ipset

STEP 6 OF 7

6

SNAT vs DNAT and masquerade

```
$ sysctl -w net.ipv4.ip_forward=1
```

Firewalls: ufw, firewalld, iptables, nftables, ipset

STEP 7 OF 7

7

Stateful inspection demo

```
$ iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

Firewalls: ufw, firewalld, iptables, nftables, ipset

Test it

ufw status and nft list ruleset show the loaded rules; conntrack states drive stateful acceptance.

OS Hardening: Permissions, SELinux, SSH, Fail2ban

LAB 16

Apply layered OS hardening from file permissions and immutability through SELinux labeling, then harden OpenSSH and add brute-force protection. Inventory SUID binaries and legacy cleartext services.

You'll build

Set special permission bits, ACLs, and immutability, demo SELinux, harden SSH, and run fail2ban.

Key commands: `chmod setuid/setgid/sticky bit, umask / chattr immutability, POSIX ACLs (setfacl/getfacl), SELinux chcon/restorecon/audit2allow, OpenSSH hardening (PermitRootLogin), fail2ban`

OS Hardening: Permissions, SELinux, SSH, Fail2ban

STEP 1 OF 7

1

chmod, chown, special bits

```
$ useradd -m alice 2>/dev/null
```

OS Hardening: Permissions, SELinux, SSH, Fail2ban

STEP 2 OF 7

2

umask, chattr, lsattr

```
$ umask 027
```


OS Hardening: Permissions, SELinux, SSH, Fail2ban

STEP 3 OF 7

3

POSIX ACLs

```
$ apt install -y acl
```

OS Hardening: Permissions, SELinux, SSH, Fail2ban

STEP 4 OF 7

4

SELinux demo inside Rocky

```
$ apt install -y docker.io && systemctl start docker
```

OS Hardening: Permissions, SELinux, SSH, Fail2ban

STEP 5 OF 7

5

Harden OpenSSH

```
$ apt install -y openssh-server
```

OS Hardening: Permissions, SELinux, SSH, Fail2ban

STEP 6 OF 7

6

fail2ban

```
$ apt install -y fail2ban
```

OS Hardening: Permissions, SELinux, SSH, Fail2ban

STEP 7 OF 7

7

Hunt SUID binaries and review legacy services

```
$ find / -perm -4000 -type f 2>/dev/null | head -n 20
```

OS Hardening: Permissions, SELinux, SSH, Fail2ban

Test it

fail2ban-client status sshd shows the active jail and sshd -t confirms the hardened config is valid.

Account Hardening

LAB 17

Harden user accounts through password quality and aging policies, brute-force lockouts, and restricted or non-interactive shells. Explore TOTP MFA and Have I Been Pwned k-anonymity lookups.

You'll build

Tune login.defs, enforce pwquality, set aging with chage, add pam_faillock, and lock down shells.

Key commands: /etc/login.defs (PASS_MAX_DAYS), pam_pwquality / pwscore, chage aging, passwd -l/-u, pam_faillock lockout, nologin / rbash restricted shells, Google Authenticator TOTP MFA

Account Hardening

STEP 1 OF 7

1

login.defs and a test user

```
$ apt update && apt install -y libpam-pwquality
```


Account Hardening

STEP 2 OF 7

2

Password quality with pwquality

```
$ minlen = 14
```

Account Hardening

STEP 3 OF 7

3

Password aging and lock/unlock

```
$ chage -M 90 -m 1 -W 7 alice
```

Account Hardening

STEP 4 OF 7

4

pam_faillock logout

```
$ deny = 5
```

Account Hardening

STEP 5 OF 7

5

Service accounts with nologin and rbash

```
$ useradd -r -s /usr/sbin/nologin svcacct
```

Account Hardening

STEP 6 OF 7

6

MFA with Google Authenticator (walkthrough)

```
$ apt install -y libpam-google-authenticator
```

Account Hardening

STEP 7 OF 7



Pwned Passwords k-anonymity lookup

```
$ apt install -y curl
```

Account Hardening

Test it

chage -l alice and passwd -S alice confirm aging and lock state; the HIBP query flags breached passwords.

Cryptography: GPG, LUKS, OpenSSL, WireGuard

LAB 18

Practice core Linux cryptography: GPG key generation and signing, LUKS2 disk encryption, OpenSSL certificates, hashing and HMAC, and WireGuard keys. Learn to identify weak TLS versions and ciphers.

You'll build

Generate GPG keys, encrypt/sign, build a LUKS2 volume, create an OpenSSL cert, and make WireGuard keys.

Key commands: GPG batch key generation, encrypt/sign, LUKS2 (cryptsetup, loopback), OpenSSL genrsa / req -x509, hashing vs HMAC, WireGuard Curve25519 keys, TLS version/cipher inspection (s_client)

Cryptography: GPG, LUKS, OpenSSL, WireGuard

STEP 1 OF 7

1

GPG key generation in batch mode

```
$ apt update && apt install -y gnupg
```

Cryptography: GPG, LUKS, OpenSSL, WireGuard

STEP 2 OF 7



2

Encrypt, decrypt, and detached-sign

```
$ echo "top secret payload" > /tmp/msg.txt
```

Cryptography: GPG, LUKS, OpenSSL, WireGuard

STEP 3 OF 7

3

LUKS2 on a loopback device

```
$ apt install -y cryptsetup
```

Cryptography: GPG, LUKS, OpenSSL, WireGuard

STEP 4 OF 7

4

OpenSSL keys and self-signed certificate

```
$ apt install -y openssl
```

Cryptography: GPG, LUKS, OpenSSL, WireGuard

STEP 5 OF 7

5

Hashing and HMAC

```
$ sha256sum /tmp/msg.txt
```

Cryptography: GPG, LUKS, OpenSSL, WireGuard

STEP 6 OF 7

6

WireGuard key pair and sample config

```
$ apt install -y wireguard-tools
```

Cryptography: GPG, LUKS, OpenSSL, WireGuard

STEP 7 OF 7

7

TLS version and cipher inspection

```
$ echo | openssl s_client -connect example.com:443 -tls1_2 2>/dev/null \
```

Cryptography: GPG, LUKS, OpenSSL, WireGuard

Test it

gpg --verify confirms the signature and gpg -d recovers plaintext; the LUKS volume mounts only with the passphrase.

Compliance, Auditing, and File Integrity

LAB 19

Establish file-integrity monitoring and run compliance and audit tooling to detect tampering, rootkits, and misconfigurations. Cover secure deletion, login banners, and OpenSCAP/CIS benchmarks.

You'll build

Baseline files with AIDE, scan with rkhunter/lynis/nmap, verify packages, and set banners.

Key commands: AIDE file-integrity baseline, rkhunter rootkit scan, debsums / rpm -V package verification, lynis system audit, nmap -sV service detection, shred secure deletion

Compliance, Auditing, and File Integrity

STEP 1 OF 7

1

AIDE baseline and tamper check

```
$ apt update && apt install -y aide
```

Compliance, Auditing, and File Integrity

STEP 2 OF 7

2

Rootkit hunting with rkhunter

```
$ apt install -y rkhunter
```

Compliance, Auditing, and File Integrity

STEP 3 OF 7

3

Package verification: debsums and rpm -V

```
$ apt install -y debsums
```

Compliance, Auditing, and File Integrity

STEP 4 OF 7

4

System audit with lynis

```
$ apt install -y lynis
```

Compliance, Auditing, and File Integrity

STEP 5 OF 7

5

nmap service detection

```
$ apt install -y nmap openssh-server
```

Compliance, Auditing, and File Integrity

STEP 6 OF 7

6

Secure deletion and wipe

```
$ echo "very sensitive" > /tmp/secret.txt
```

Compliance, Auditing, and File Integrity

STEP 7 OF 7

7

Banners and OpenSCAP / CIS pointers

```
$ echo "Authorized use only. Activity is monitored." > /etc/issue
```


Compliance, Auditing, and File Integrity

Test it

aide --check flags the tampered /etc/hostname, proving the integrity baseline detects unauthorized change.

Recap — Security

- Lab 14: AAA: sudo, PAM, and Polkit
- Lab 15: Firewalls: ufw, firewalld, iptables, nftables, ipset
- Lab 16: OS Hardening: Permissions, SELinux, SSH, Fail2ban
- Lab 17: Account Hardening
- Lab 18: Cryptography: GPG, LUKS, OpenSSL, WireGuard
- Lab 19: Compliance, Auditing, and File Integrity

Lunch Break

1 hour

 DOMAIN 04

Automation, Orchestration, and Scripting

Objectives 4.1–4.5 · 17% of the exam · Labs 20–24

Key Concepts — Automation, Orchestration, and Scripting

1

Infrastructure as Code (Ansible, Puppet, OpenTofu) makes configuration repeatable and idempotent.

2

Bash scripting automates tasks with variables, conditionals, loops, functions and shellcheck-clean code.

3

Python (venv, modules, PEP 8) extends automation beyond the shell; Git version-controls all of it.

4

AI assistants are used responsibly: verify-before-paste, redact secrets, and follow a corporate AI policy.

Hands-On Labs — Automation, Orchestration, and Scripting

Labs 20–21

- Infrastructure as Code with Ansible
- Bash Scripting

Labs 22–23

- Python for System Administration
- Git Version Control

Lab 24

- Responsible AI Use for Linux Administrators

Infrastructure as Code with Ansible

LAB 20

Learn Infrastructure as Code using Ansible as the primary agentless configuration management tool, and recognize the wider IaC ecosystem (Puppet, OpenTofu, cloud-init, Kubernetes ConfigMaps). Build an idempotent playbook that deploys and configures nginx.

You'll build

Install Ansible, write an inventory, run ad-hoc commands, and author an idempotent playbook with variables, templates, and handlers.

Key commands: Ansible, inventory, playbook, idempotency, handlers/templates, ansible-galaxy

Infrastructure as Code with Ansible

STEP 1 OF 6

1

Install Ansible and prepare workspace

```
$ apt update
```


Infrastructure as Code with Ansible

STEP 2 OF 6

2

Create inventory pointing at localhost

```
$ [web]
```

Infrastructure as Code with Ansible

STEP 3 OF 6

3

Run ad-hoc commands

```
$ cd /root/lab
```

Infrastructure as Code with Ansible

STEP 4 OF 6

4

Write a playbook with variable, template, and handler

```
$ - name: Deploy a simple nginx site
```

Infrastructure as Code with Ansible

STEP 5 OF 6

5

Install a Galaxy collection and peek at other IaC tools

```
$ ansible-galaxy collection install community.general
```

Infrastructure as Code with Ansible

STEP 6 OF 6

6

GitOps, cloud-init, and Kubernetes ConfigMap samples

```
$ package_update: true
```

Infrastructure as Code with Ansible

Test it

Idempotent playbooks converge state without unnecessary restarts; Ansible, Puppet, OpenTofu, cloud-init, and ConfigMaps each occupy a distinct IaC niche.

Bash Scripting

LAB 21

Practice core Bash scripting constructs (variables, conditionals, loops, functions, option parsing, regex, file tests) and assemble a defensive backup script, then audit it with shellcheck.

You'll build

Write a series of Bash scripts exercising each language feature, build a real /etc backup script, and lint everything with shellcheck.

Key commands: shebang, parameter expansion, arrays, test operators `[[ ]]`, getopt, IFS

Bash Scripting

STEP 1 OF 7

1

Set up a scripts directory

```
$ mkdir -p /root/scripts && cd /root/scripts
```


Bash Scripting

STEP 2 OF 7

2

Variables, command substitution, parameter expansion

```
$ name="Linux+"
```

Bash Scripting

STEP 3 OF 7

3

Conditionals, case, and test operators

```
$ num=${1:-0}
```

Bash Scripting

STEP 4 OF 7

4

Loops, functions, and exit codes

```
$ log() { echo "[$(date +%T)] $*"; }
```

Bash Scripting

STEP 5 OF 7

5

getopts and IFS-based CSV parsing

```
$ verbose=0
```

Bash Scripting

STEP 6 OF 7

6

Build a real backup script

```
$ set -euo pipefail
```

Bash Scripting

STEP 7 OF 7

7

Lint with shellcheck

```
$ shellcheck /root/scripts/*.sh || true
```

Bash Scripting

Test it

A robust script uses `set -euo pipefail`, quotes variables, and passes shellcheck cleanly.

Python for System Administration

LAB 22

Use Python for common Linux admin tasks: create a virtual environment, exercise core data types, walk the filesystem, parse JSON, call a REST API with requests, and enforce PEP 8 with black and flake8.

You'll build

Set up a venv, write scripts covering data types, `os.walk`, JSON parsing, and HTTP requests, then organize code into a package and lint it.

Key commands: `venv/pip`, core types (`bool/int/float/str/list/dict`), type hints, `os.walk`, `json` module, `requests`

Python for System Administration

STEP 1 OF 7

1

Check Python and create a virtual environment

```
$ python3 --version
```

Python for System Administration

STEP 2 OF 7

2

Install dependencies

```
$ pip install requests black flake8
```

Python for System Administration

STEP 3 OF 7

3

Exercise core data types

```
$ """Demonstrate the built-in Python data types."""
```

Python for System Administration

STEP 4 OF 7

4

Walk /etc, count files, emit JSON

```
$ ""Walk /etc and print a JSON summary.""
```

Python for System Administration

STEP 5 OF 7

5

Parse a JSON file with the json module

```
${
```

Python for System Administration

STEP 6 OF 7

6

Call a public API with requests

```
$ """Call a public test API and print selected fields."""
```

Python for System Administration

STEP 7 OF 7

7

Use a package module and apply PEP 8

```
$ mkdir -p ~/pylab/helpers
```

Python for System Administration

Test it

A venv isolates dependencies, requests loudly fails on bad HTTP, and black+flake8 enforce PEP 8 automatically.

Git Version Control

LAB 23

Work through Git end to end: init, commit, branch, merge, resolve conflicts, stash, tag, rewrite history with reset and rebase, and push/pull against a local bare remote.

You'll build

Create a working repo and a bare origin repo, then practice the full Git workflow from first commit through conflict resolution, rebasing, and remote push/pull.

Key commands: `git init/config`, `branch/merge --no-ff`, `conflict resolution`, `stash/tag`, `reset --soft/rebase`, `bare remote`

Git Version Control

STEP 1 OF 7

1

Install Git and set identity

```
$ apt update && apt install -y git
```

Git Version Control

STEP 2 OF 7

2

Initialise a repo and make the first commit

```
$ mkdir -p /root/lab && cd /root/lab
```

Git Version Control

STEP 3 OF 7

3

Branch, modify, merge

```
$ git switch -c feature/banner
```

Git Version Control

STEP 4 OF 7

4

Create and resolve a conflict

```
$ git switch -c feature/edit
```

Git Version Control

STEP 5 OF 7

5

diff, stash, tag

```
$ echo "draft note" >> README.md
```

Git Version Control

STEP 6 OF 7

6

Rewrite history: reset and rebase

```
$ echo "oops" > scratch.txt
```

Git Version Control

STEP 7 OF 7

7

Add a remote and push/pull

```
$ git init --bare /tmp/origin.git
```


Git Version Control

Test it

Git tracks history through commits and branches; merge/rebase integrate work and pull --rebase keeps a linear remote history.

Responsible AI Use for Linux Administrators

LAB 24

Adopt a verify-before-paste workflow for AI-generated Linux commands: redact secrets before sending prompts to hosted models, lint and sandbox suggestions, weigh local vs hosted models, and draft a corporate AI usage policy.

You'll build

Build a verification workspace, redact secrets in prompts, shellcheck and sandbox an AI-suggested command, and author a verification checklist plus an AI usage policy.

Key commands: verify-before-paste, secret redaction, local vs hosted LLMs, shellcheck, API key via env var, AI usage policy

Responsible AI Use for Linux Administrators

STEP 1 OF 7

1

Set up a verification workspace

```
$ apt update && apt install -y shellcheck jq curl
```

Responsible AI Use for Linux Administrators

STEP 2 OF 7

2

Try a local LLM (optional)

```
$ curl -fsSL https://ollama.com/install.sh | sh || echo "Ollama install skipped"
```

Responsible AI Use for Linux Administrators

STEP 3 OF 7

3

Hosted-API placeholder workflow with redaction

```
$ export LLM_API_KEY="REDACTED-EXAMPLE"
```

Responsible AI Use for Linux Administrators

STEP 4 OF 7

4

Verify an AI-suggested command

```
$ find /var/log -type f -printf '%s %p\n' 2>/dev/null \
```

Responsible AI Use for Linux Administrators

STEP 5 OF 7

5

Build a verification checklist

```
$ - [ ] Did I read every line of the suggested command/script?
```

Responsible AI Use for Linux Administrators

STEP 6 OF 7

6

Draft a corporate AI usage policy

\$ - Local models on company hardware: allowed for internal, non-secret data.

Responsible AI Use for Linux Administrators

STEP 7 OF 7

7

Compare local vs hosted at a glance

\$ Local model (Ollama, Llamafire)

Responsible AI Use for Linux Administrators

Test it

Always verify before paste: redact secrets, shellcheck, sandbox, then run; choose local vs hosted models per policy.

Recap — Automation, Orchestration, and Scripting

- Lab 20: Infrastructure as Code with Ansible
- Lab 21: Bash Scripting
- Lab 22: Python for System Administration
- Lab 23: Git Version Control
- Lab 24: Responsible AI Use for Linux Administrators

Tea Break

15 minutes

 DOMAIN 05

Troubleshooting

Objectives 5.1–5.5 · 22% of the exam · Labs 25–30

Key Concepts — Troubleshooting

1

Monitoring vocabulary (SLA/SLI/SLO) and tools (SNMP, Prometheus, node_exporter) establish a baseline.

2

Troubleshooting is a loop: baseline → reproduce → identify with the right counter → remediate → verify.

3

Storage/OS faults (ENOSPC, inode exhaustion, failed units) and network faults (DNS, routing, MTU) each have a signature.

4

Security faults (SELinux, ACLs, certs, lockout) and performance faults (CPU, memory, I/O) close out the domain.

Hands-On Labs — Troubleshooting

Labs 25–26

- Monitoring Concepts and Tools
- Troubleshooting Hardware, Storage, and OS Issues

Labs 27–28

- Troubleshooting Network Connectivity
- Troubleshooting Security Issues

Labs 29–30

- Troubleshooting Performance Issues
- Capstone: End-to-End Server Build and Triage

Monitoring Concepts and Tools

LAB 25

Learn the monitoring vocabulary (SLA/SLO/SLI) and stand up a working monitoring stack to produce metrics, alerts, webhooks, health checks, and aggregated logs.

You'll build

Install SNMP, node_exporter, Prometheus and Alertmanager, then write and validate alert rules, test webhooks, and inspect journald logs.

Key commands: SLA vs SLO vs SLI, SNMP OID/MIB walk, Prometheus scrape config, node_exporter metrics endpoint, promtool check rules, webhook alerting

Monitoring Concepts and Tools

STEP 1 OF 7

1

Install SNMP and walk the system MIB

```
$ apt update
```

Monitoring Concepts and Tools

STEP 2 OF 7

2

Install and probe node_exporter

```
$ apt install -y prometheus-node-exporter
```

Monitoring Concepts and Tools

STEP 3 OF 7

3

Install Prometheus and scrape node_exporter

```
$ apt install -y prometheus
```

Monitoring Concepts and Tools

STEP 4 OF 7

4

Write and validate an alert rule

```
$ groups:
```

Monitoring Concepts and Tools

STEP 5 OF 7

5

Simulate a webhook receiver

```
$ (nc -l -p 8000 -q 1 >/tmp/webhook.log 2>&1 &) ; sleep 1
```

Monitoring Concepts and Tools

STEP 6 OF 7



6

Health check pattern

```
$ curl -fsS http://localhost:9100/metrics >/dev/null && echo "node_exporter OK"
```

Monitoring Concepts and Tools

STEP 7 OF 7

7

Log aggregation via journalctl

```
$ journalctl -u prometheus -n 20 --no-pager
```

Monitoring Concepts and Tools

Test it

node_exporter, Prometheus, and its health/ready endpoints all respond, and promtool confirms the alert rule is valid.

Troubleshooting Hardware, Storage, and OS Issues

LAB 26

Deliberately inject common OS and storage failures then diagnose and repair each, keeping every breakage contained to loopback devices, throwaway units, or subshells.

You'll build

Reproduce a full filesystem, inode exhaustion, GRUB validation, a runaway CPU process, a failed systemd unit, a broken PATH, and a memory leak, then fix them.

Key commands: ENOSPC / df -h, inode exhaustion / df -i, grub-mkconfig validation, runaway process / top -bn1, failed systemd unit, broken PATH recovery

Troubleshooting Hardware, Storage, and OS Issues

STEP 1 OF 7

1

Break: fill a filesystem (ENOSPC)

```
$ mkdir -p /mnt/tinyfs
```

Troubleshooting Hardware, Storage, and OS Issues

STEP 2 OF 7

2

Break: inode exhaustion

```
$ df -i /mnt/tinyfs
```

Troubleshooting Hardware, Storage, and OS Issues

STEP 3 OF 7

3

Validate a GRUB config without committing it

```
$ grub-mkconfig -o /tmp/grub.cfg.new 2>&1 | tail -n 5
```

Troubleshooting Hardware, Storage, and OS Issues

STEP 4 OF 7

4

Break: runaway CPU process

```
$ bash -c 'while ;; do ;; done' &
```

Troubleshooting Hardware, Storage, and OS Issues

STEP 5 OF 7

5

Break: a failing systemd unit

\$ [Unit]

Troubleshooting Hardware, Storage, and OS Issues

STEP 6 OF 7

6

Break and recover a broken PATH

```
$ ( export PATH=/nonexistent
```

Troubleshooting Hardware, Storage, and OS Issues

STEP 7 OF 7

7

Simulate a memory leak

```
$ data = []
```


Troubleshooting Hardware, Storage, and OS Issues

Test it

Each fault is confirmed by the right counter (df, df -i, top, systemctl status, ps rss) and reversed cleanly without touching the host.

Troubleshooting Network Connectivity

LAB 27

Break and repair the common network faults (DNS, routing, firewall, MTU, IP, link, dual-stack) while practicing the standard L3/L4 triage toolkit.

You'll build

Baseline the host, then inject DNS, gateway, firewall, MTU, duplicate-IP, and link-down faults, and finish with IPv4/IPv6 and packet-capture triage.

Key commands: `ip addr / ip route baseline, resolv.conf / dig, default gateway, ufw port block, MTU / ping -M do, duplicate IP / arping`

Troubleshooting Network Connectivity

STEP 1 OF 7

1

Baseline and install tools

```
$ apt update
```

Troubleshooting Network Connectivity

STEP 2 OF 7

2

Break: DNS, then fix

```
$ cp /etc/resolv.conf /tmp/resolv.conf.bak
```

Troubleshooting Network Connectivity

STEP 3 OF 7

3

Break: default gateway, then revert

```
$ ORIG=$(ip route show default)
```

Troubleshooting Network Connectivity

STEP 4 OF 7

4

Break: firewall blocks a port

```
$ ufw --force enable
```

Troubleshooting Network Connectivity

STEP 5 OF 7

5

Break: MTU mismatch

```
$ IF=$(ip -br link | awk '$1!="lo" && $2=="UP" {print $1; exit}')
```

Troubleshooting Network Connectivity

STEP 6 OF 7

6

Break: duplicate IP and link down

```
$ IF=$(ip -br link | awk '$1!="lo" && $2=="UP" {print $1; exit}')
```


Troubleshooting Network Connectivity

STEP 7 OF 7

7

IPv4 vs IPv6 and capture

```
$ curl -4 -s --max-time 3 https://ifconfig.co || true
```

Troubleshooting Network Connectivity

Test it

Each single-variable fault produces its expected symptom and reverts cleanly, and IPv4/IPv6 plus capture tools confirm reachability.

Troubleshooting Security Issues

LAB 28

Diagnose and repair security faults spanning SELinux contexts, POSIX ACLs, certificate trust, account lockout, broken repos, and weak TLS, using a Rocky container for the SELinux portion.

You'll build

Pull a Rocky container for SELinux practice, then break and fix ACLs, CA trust, pam_faillock lockout, a bad apt repo, and weak TLS on the Ubuntu host.

Key commands: SELinux context / chcon / restorecon, AVC denials / ausearch / audit2allow, POSIX ACL / getfacl / setfacl, CA trust / update-ca-certificates, pam_faillock lockout, apt repo / TLS s_client

Troubleshooting Security Issues

STEP 1 OF 7

1

Prep host and pull a Rocky container

```
$ apt update
```

Troubleshooting Security Issues

STEP 2 OF 7

2

Break: SELinux context, then restorecon

```
$ docker exec selab bash -c '
```

Troubleshooting Security Issues

STEP 3 OF 7

3

Inspect AVC denials with audit2allow

```
$ docker exec selab bash -c '
```

Troubleshooting Security Issues

STEP 4 OF 7

4

Break: ACL on Ubuntu host

```
$ useradd -m alice 2>/dev/null || true
```

Troubleshooting Security Issues

STEP 5 OF 7

5

Break: certificate trust

```
$ mkdir -p /tmp/ca && cd /tmp/ca
```


Troubleshooting Security Issues

STEP 6 OF 7

6

Break: account lockout with pam_faillock

```
$ auth required pam_faillock.so preauth deny=3 unlock_time=120
```

Troubleshooting Security Issues

STEP 7 OF 7

7

Break: misconfigured apt repo and weak TLS

```
$ echo 'deb http://no.such.host.invalid/ubuntu jammy main' > /etc/apt/sources.list.d/broke
```

Troubleshooting Security Issues

Test it

restorecon fixes SELinux type, getfacl/setfacl explains hidden ACL denials, the CA trust bundle rebuilds, faillock resets, and modern TLS negotiates while TLS1.0 is refused.

Troubleshooting Performance Issues

LAB 29

Stress CPU, memory, disk I/O, and network in turn, then walk the universal performance loop: baseline, spike, identify the bottleneck with the right counter, and resolve it.

You'll build

Install stress-ng, fio, and sysstat, baseline the host, then generate controlled CPU, memory, disk, and network spikes and attribute each to the right counter and PID.

Key commands: baseline: uptime/free/vmstat/mpstat, stress-ng CPU spike / load average, memory pressure / vmstat si-so, fio disk storm / iostat -x, D-state / ping jitter, pidstat context switches

Troubleshooting Performance Issues

STEP 1 OF 7

1

Install the toolkit and baseline

```
$ apt update
```

Troubleshooting Performance Issues

STEP 2 OF 7

2

Generate and observe a CPU spike

```
$ stress-ng --cpu 4 --timeout 30 &
```

Troubleshooting Performance Issues

STEP 3 OF 7

3

Memory pressure and swap behavior

```
$ free -m
```

Troubleshooting Performance Issues

STEP 4 OF 7

4

Disk I/O storm

```
$ fio --name=lab --rw=randwrite --bs=4k --size=128M --runtime=20 --time_based \
```


Troubleshooting Performance Issues

STEP 5 OF 7

5

Network jitter and blocked processes

```
$ ping -i 0.2 -c 20 8.8.8.8 | tail -n 5
```

Troubleshooting Performance Issues

STEP 6 OF 7

6

Context switches and interactive tools

```
$ pidstat -w 1 5
```

Troubleshooting Performance Issues

STEP 7 OF 7

7

Walk the full loop on a synthetic incident

```
$ echo "BASELINE: "; uptime; free -m | head -n 2
```

Troubleshooting Performance Issues

Test it

Each spike maps to its diagnostic counter (load vs mpstat, si/so vs free, %util vs await) and the triage script shows baseline-to-recovery.

Capstone: End-to-End Server Build and Triage

LAB 30

Integrate Objectives 1-5 into one coherent build: a hardened admin account, LVM storage, nginx with a systemd override, a locked firewall, a health-check timer, and git-versioned config, then inject a fault and triage it end-to-end.

You'll build

Provision webops with hardened SSH, build LVM-backed /srv/web, deploy nginx, lock ufw to web ports, add a health-check timer, version /etc/nginx in git, then inject and remediate a disk-full fault.

Key commands: hardened SSH / narrow sudo, LVM: pvcreate/vgcreate/lvcreate, systemd drop-in override, ufw default-deny allowlist, systemd timer health check, git-versioned /etc/nginx

Capstone: End-to-End Server Build and Triage

STEP 1 OF 7

1

Provision a user and harden SSH

```
$ apt update
```

Capstone: End-to-End Server Build and Triage

STEP 2 OF 7



2

Build LVM-backed /srv/web over loopback

```
$ dd if=/dev/zero of=/var/lab-pv.img bs=1M count=200
```

Capstone: End-to-End Server Build and Triage

STEP 3 OF 7

3

Install nginx with a systemd override binding /srv/web

```
$ rm -f /etc/nginx/sites-enabled/default
```


Capstone: End-to-End Server Build and Triage

STEP 4 OF 7

4

Firewall: allow only 80/443 and SSH

```
$ ufw --force reset >/dev/null
```

Capstone: End-to-End Server Build and Triage

STEP 5 OF 7

5

Health-check script with a systemd timer

```
$ set -euo pipefail
```

Capstone: End-to-End Server Build and Triage

STEP 6 OF 7

6

Commit /etc/nginx to a local git repo

```
$ cd /etc/nginx
```

Capstone: End-to-End Server Build and Triage

STEP 7 OF 7

7

Inject a fault and triage

```
$ systemctl stop nginx
```

Capstone: End-to-End Server Build and Triage

Test it

After remediation curl returns the Capstone page with a 200, confirming the full build and the status/journal/sockets/storage/probe triage loop.

Recap — Troubleshooting

- Lab 25: Monitoring Concepts and Tools
- Lab 26: Troubleshooting Hardware, Storage, and OS Issues
- Lab 27: Troubleshooting Network Connectivity
- Lab 28: Troubleshooting Security Issues
- Lab 29: Troubleshooting Performance Issues
- Lab 30: Capstone: End-to-End Server Build and Triage



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

System Management

Boot & FHS, kernel modules, LVM storage, networking, shell, backup, virtualization.

2

Services & User Management

Files, accounts, processes, packages, systemd services, containers.

3

Security

AAA/sudo/PAM, firewalls, OS & account hardening, cryptography, compliance & audit.

4

Automation & Scripting

Ansible IaC, Bash, Python, Git version control, responsible AI use.

5

Troubleshooting

Monitoring, and storage / network / security / performance triage.

Preparing for the Linux+ XK0-006 Exam

- Redo every lab on Killercode until the commands are automatic.
- Review the 'Test it' takeaway in each lab and the Learner Guide.
- Know which tool solves which problem — the exam is scenario-based (performance-based items).
- The exam has up to 90 questions (multiple-choice + performance-based) in 90 minutes.
- Book the exam through Pearson VUE from your CompTIA account.

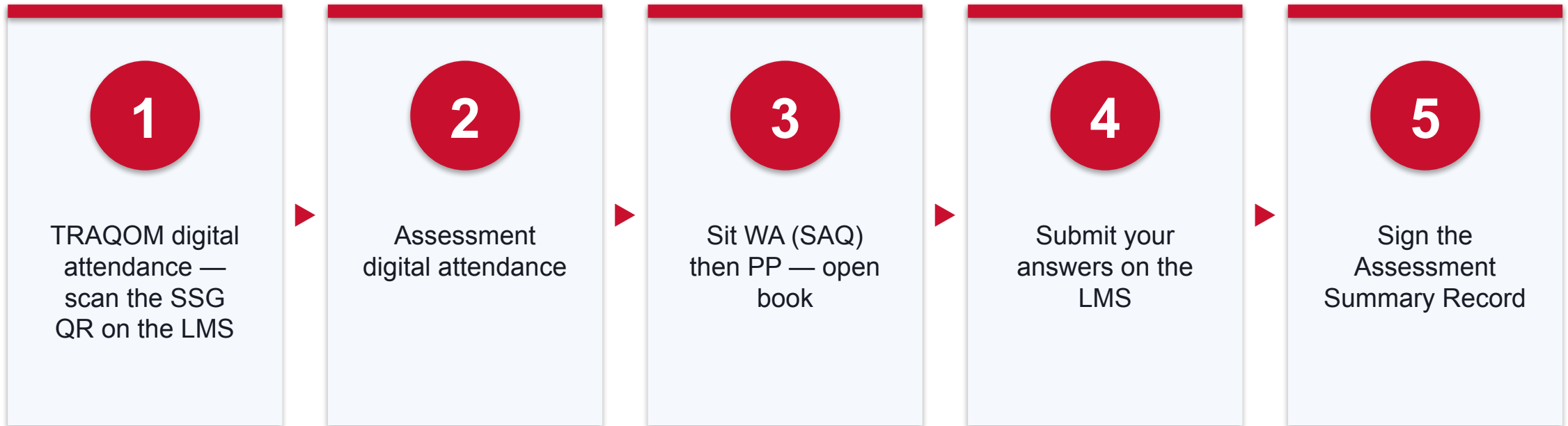
Practice Exam

- Sharpen your exam readiness with the Tertiary Infotech CompTIA Linux+ practice exam.
- Practice exam: <https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-linux-plus>
- Attempt it under timed conditions and review every explanation.
- Revisit any lab whose domain you miss, then re-take the practice exam.

Assessment

- Written Assessment (SAQ) — 1 hour. Practical Performance (PP) — 1 hour.
- Open book: slides, Learner Guide and approved materials only.
- Remember to take the Assessment digital attendance (TRAQOM · SSG).
- Submit your completed answers on the LMS at <https://lms-tms.tertiaryinfotech.com/>.

Assessment Flow



Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

ALL THE BEST

Thank You!

You are now ready to administer Linux — and to sit the CompTIA Linux+ XK0-006 exam.